字符串算法竞赛板子

本板子默认 C++17。字符串位置、区间和返回的匹配位置均从 `0` 开始；区间统一写成左闭右开 `[l,r)`。代码以赛场直接复制为目标：变量名短、接口集中，代码块内只保留传参和关键边界的短注释，完整说明放在代码块外。所有 64 位有符号整数统一写作 `ll`。

0. 公共约定与选型

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
```

问题	推荐算法	时间复杂度
单模式串匹配、周期、Border	KMP / 前缀函数	`O(n+m)`
字符串 DP 中维护匹配长度	KMP 自动机	预处理 `O(n|Sigma|)`
一个串与所有后缀的 LCP	Z 函数	`O(n)`
模式与文本每个后缀的 LCP	扩展 KMP	`O(n+m)`
循环同构、最小循环移位	Booth 最小表示法	`O(n)`
多次比较任意子串、判回文	字符串哈希	预处理 `O(n)`，单次 `O(1)`
单点修改后比较/判回文	动态双向哈希	单次 `O(log n)`
所有回文中心、最长回文子串	Manacher	`O(n)`
前缀查询、字典计数	Trie	总字符数线性
多模式串匹配	AC 自动机	总字符数线性加输出量
静态后缀排序、任意后缀 LCP	后缀数组 SA	`O(n log n)`，LCP 查询 `O(1)`
超长字符串线性后缀排序	SA-IS	`O(n+|Sigma|)`
本质不同子串、子串出现次数	后缀自动机 SAM	`O(n|Sigma|)`；小写字母即 `O(n)`
本质不同回文串、回文出现次数	回文自动机 PAM	`O(n|Sigma|)`；小写字母即 `O(n)`
字典序分解、最小后缀类问题	Lyndon / Duval	`O(n)`
判断是否为子序列、大量询问	子序列自动机	预处理 `O(n|Sigma|)`，询问 `O(m)`
两串 LCS	滚动 DP / bitset LCS	`O(nm)` / `O(nm/64)`
相同字符对很少的 LCS	Hunt-Szymanski	`O((n+r)log m)`

问题	推荐算法	时间复杂度
编辑距离	滚动 DP	`O(nm)` 时间、`O(m)` 空间
较短串不超过 64 的编辑距离	Myers 位并行	`O(max(n,m))`
最大前后缀重叠、顺序拼接	KMP 重叠	总长度线性
少量字符串的最短公共超串	子集 DP + 重叠	`O(n^2 2^n)`
最少回文划分	Manacher + DP	`O(n^2)`
不同子序列计数	末次出现 DP	`O(n)`

`|Sigma|` 表示字符集大小。固定小字符集时二维数组最快；字符集很大时应离散化后用 `vector<pair<int,int>>`、`map` 或 `unordered_map` 存转移，不能直接开巨大的二维数组。

---

1. KMP 与前缀函数

1.1 前缀函数

`pi[i]` 是 `s[0..i]` 的最长真前缀长度，且该前缀同时是它的后缀。所谓真前缀是不等于整个串的前缀。

它可用于求 Border、周期、循环节、单模式串匹配，以及统计每个前缀出现次数。时间 `O(n)`，空间 `O(n)`。

```
vector<int> prefix(const string& s) { // s: 待处理字符串
    int n = s.size();
    vector<int> p(n);
    for (int i = 1; i < n; i++) {
        int j = p[i - 1];
        while (j && s[i] != s[j]) j = p[j - 1];
        if (s[i] == s[j]) j++;
        p[i] = j;
    }
    return p;
}
```

空串会返回空数组。`p[i]` 是长度而不是下标，所以失配时跳到 `p[j-1]`。

1.2 KMP 查找全部出现位置

`kmp(s,t)` 返回模式串 `t` 在文本串 `s` 中所有出现的起点，允许重叠。例如 `kmp("aaaa","aa")` 返回 `{0,1,2}`。

- `s`：文本串。
- `t`：模式串，要求非空。
- 返回：所有 `0-based` 起点，按递增顺序。

- 复杂度：时间 `O(|s|+|t|)`，空间 `O(|t|)`。

```
vector<int> kmp(const string& s, const string& t) { // s: 文本; t: 非空模式
    vector<int> p = prefix(t), a;
    for (int i = 0, j = 0; i < (int)s.size(); i++) {
        while (j && s[i] != t[j]) j = p[j - 1];
        if (s[i] == t[j]) j++;
        if (j == (int)t.size()) {
            a.push_back(i - j + 1);
            j = p[j - 1];
        }
    }
    return a;
}
```

若题目约定空模式串匹配所有位置，应在调用前单独处理；竞赛题通常不会给空模式串。

1.3 Border、周期与循环节

串 `s` 的 Border 是既为前缀又为后缀的真子串。沿 `pi[n-1]` 不断跳转即可得到全部 Border 长度。

```
vector<int> borders(const string& s) { // 返回所有非空 Border 长度，递增
    vector<int> p = prefix(s), a;
    for (int k = s.empty() ? 0 : p.back(); k; k = p[k - 1]) a.push_back(k);
    reverse(a.begin(), a.end());
    return a;
}
```

令 `d=n-pi[n-1]`：

- `d` 始终是最小周期，即对所有合法 `i` 都有 `s[i]=s[i-d]`；末尾可以是不完整的一段。
- 只有当 `n%d==0` 时，`d` 才是最小循环节长度；否则最小循环节长度是 `n`。
- 空串没有这里定义的周期，需单独处理。

```
int min_period(const string& s) { // s: 非空; 允许末段不完整
    auto p = prefix(s);
    return (int)s.size() - p.back();
}

int min_block(const string& s) { // s: 非空; 要求整串由循环节重复得到
    int n = s.size(), d = min_period(s);
    return n % d == 0 ? d : n;
}
```

1.4 每个前缀在整串中的出现次数

返回 `c[len]`，表示长度为 `len` 的前缀在 `s` 中作为子串出现多少次，包含它作为自身前缀的那次。适合“每个 Border 出现几次”等问题。

```
vector<int> prefix_cnt(const string& s) { // c[len]: 长为 len 的前缀出现次数
    int n = s.size();
    auto p = prefix(s);
    vector<int> c(n + 1);
    for (int x : p) c[x]++;
    for (int i = n; i > 0; i--) c[p[i - 1]] += c[i];
    for (int i = 0; i <= n; i++) c[i]++;
    return c;
}
```

时间和空间均为 `O(n)`。`c[0]=n+1` 表示空串在字符间隙中的出现次数，一般忽略。

1.5 所有周期与 KMP 自动机

若 `b` 是长度 `n` 的串的 Border 长度，则 `n-b` 是它的一个周期；此外 `n` 本身总是周期。因此全部周期可由 Border 链得到：

```
vector<int> periods(const string& s) { // 返回所有周期长度，递增；s 非空
    int n = s.size();
    auto b = borders(s);
    vector<int> a;
    for (int x : b) a.push_back(n - x);
    a.push_back(n);
    sort(a.begin(), a.end());
    return a;
}
```

这里的周期允许最后一段不完整；若要求整串恰好由某段重复组成，还需筛选 `n%d==0`。

Fine-Wilf 定理：若长度为 `n` 的串同时有周期 `p,q`，且 `n>=p+q-gcd(p,q)`，则 `gcd(p,q)` 也是它的周期。它常用于证明两个周期能否合并；长度条件不足时结论不一定成立。

KMP 自动机维护“当前文本后缀与模式前缀的最大匹配长度”。常用于数位/状压 DP 中统计“不出现禁串”“恰好出现若干次”等问题。状态 `j` 的范围是 `0..m`；`go[j][c]` 表示追加字符 `c` 后的新匹配长度。到达 `m` 表示刚匹配完整模式，之后是否退回 `pi[m-1]` 由题目 DP 决定。

```
vector<array<int,26>> kmp_go(const string& s) { // s: 非空小写模式串
    int n = s.size();
    auto p = prefix(s);
    vector<array<int,26>> go(n + 1);
    for (int c = 0; c < 26; c++) go[0][c] = (s[0] - 'a' == c);
    for (int i = 1; i <= n; i++) {
        int f = i == n ? p[n - 1] : p[i - 1];
        for (int c = 0; c < 26; c++) {
            if (i < n && s[i] - 'a' == c) go[i][c] = i + 1;
```

```
            else go[i][c] = go[f][c];
        }
    }
    return go;
}
```

预处理 `O(26m)`，每次转移 `O(1)`。若匹配到完整模式后允许重叠继续匹配，处理一次命中后把 DP 状态改为 `pi[m-1]`；若状态 `m` 本身代表"已经出现过"，则可把它设为吸收态。

---

2. Z 函数（扩展 KMP）

`z[i]` 是 `s` 与后缀 `s[i..n)` 的最长公共前缀长度。本模板约定 `z[0]=n`。它适合求每个位置与整串前缀的匹配长度、模式匹配、周期判定和字符串拼接约束。

```
vector<int> zfunc(const string& s) { // z[i]=lcp(s,s[i..])
    int n = s.size();
    vector<int> z(n);
    if (!n) return z;
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
```

时间 `O(n)`，空间 `O(n)`。这里维护的是半开 Z-box `[l,r)`，因此条件是 `i<r`。

2.1 用 Z 函数匹配模式串

把 `t + 分隔符 + s` 拼起来，后半部分 Z 值至少为 `|t|` 的位置就是匹配点。分隔符必须不出现在两串中；下面不依赖字符分隔符，直接对整数数组加入 `-1`，因此任意字节字符串都可用。

```
vector<int> zmatch(const string& s, const string& t) { // s: 文本; t: 非空模式
    vector<int> a;
    for (unsigned char c : t) a.push_back(c);
    a.push_back(-1);
    for (unsigned char c : s) a.push_back(c);
    int n = a.size(), m = t.size();
    vector<int> z(n), ans;
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && a[z[i]] == a[i + z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    for (int i = m + 1; i < n; i++)
        if (z[i] >= m) ans.push_back(i - m - 1);
```

```
    return ans;
}
```

复杂度 `O(|s|+|t|)`。只做普通匹配时 KMP 更省内存；已经需要 LCP 信息时 Z 函数更自然。

2.2 扩展 KMP：模式与文本每个后缀的 LCP

`exkmp(s,t)[i]` 表示文本后缀 `s[i..)` 与模式串 `t` 的 LCP 长度。与拼接后做一次 Z 函数等价，但无需寻找分隔符，也不会建立拼接串。

- `s`：文本串，可以为空。
- `t`：模式串，可以为空；为空时所有答案为 `0`。
- 返回长度为 `|s|` 的数组。
- 时间 `O(|s|+|t|)`，空间 `O(|s|+|t|)`。

```
vector<int> exkmp(const string& s, const string& t) { // ans[i]=lcp(s[i..],t)
    int n = s.size(), m = t.size();
    vector<int> z = zfunc(t), a(n);
    if (!m) return a;
    for (int i = 0, l = 0, r = 0; i < n; i++) {
        if (i < r) a[i] = min(r - i, z[i - l]);
        while (a[i] < m && i + a[i] < n && t[a[i]] == s[i + a[i]]) a[i]++;
        if (i + a[i] > r) l = i, r = i + a[i];
    }
    return a;
}
```

模式串在 `i` 出现当且仅当 `a[i]==|t|`。相比只返回匹配位置，它还能直接回答每个位置最多匹配了多少字符。

---

3. 最小表示法（Booth）

求字符串所有循环移位中字典序最小者的起点。用于循环同构、项链问题、规范化环形序列。若最小循环移位不唯一，返回其中最小的起点。

- `booth(s)`：返回最小循环移位的 `0-based` 起点；空串返回 `0`。
- `min_rot(s)`：返回最小循环移位后的字符串。
- 时间 `O(n)`，额外空间 `O(n)`（由 `s+s` 产生）。

```
int booth(const string& s) { // 返回字典序最小循环移位起点
    int n = s.size();
    if (!n) return 0;
    string t = s + s;
    int i = 0, j = 1, k = 0;
    while (i < n && j < n) {
        k = 0;
        while (k < n && t[i + k] == t[j + k]) k++;
```

```
        if (k == n) break;
        if (t[i + k] > t[j + k]) i += k + 1;
        else j += k + 1;
        if (i == j) j++;
    }
    return min(i, j);
}

string min_rot(const string& s) { // 返回最小循环移位串
    if (s.empty()) return s;
    int k = booth(s);
    return s.substr(k) + s.substr(0, k);
}
```

判断两个等长串是否循环同构，可比较 `min_rot(a)==min_rot(b)`；只做一次判断也可 KMP 搜索 `b` 是否出现于 `a+a` 的前 `n` 个起点。

---

4. 高性能字符串哈希

哈希适合大量静态子串相等判断、最长公共前缀二分、判回文和子串排序。它是概率算法，存在极低碰撞概率；要求绝对正确时使用后缀数组、Trie 或直接比较。

下面使用 `ull` 自然溢出模 `2^64`，速度快。基数在运行时随机选择，并把字符映射为 `1..256`，避免前导零问题。一次程序内需要比较的所有字符串必须共用同一个 `BASE`。

4.1 静态前缀哈希

`Hash h(s)` 预处理字符串。`get(l,r)` 返回 `[l,r)` 的哈希值，空串哈希为 `0`。

```
static const ull BASE = [] {
    ull x = chrono::steady_clock::now().time_since_epoch().count();
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    x ^= x >> 31;
    return x | (1ULL << 32) | 1;
}();

struct Hash {
    vector<ull> h, p;
    Hash() {}
    Hash(const string& s) { init(s); }
    void init(const string& s) { // s: 原串
        int n = s.size();
        h.assign(n + 1, 0), p.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = h[i] * BASE + (unsigned char)s[i] + 1;
            p[i + 1] = p[i] * BASE;
        }
    }
    ull get(int l, int r) const { // 子串 [l,r)
```

```
        return h[r] - h[l] * p[r - l];
    }
};
```

预处理 `O(n)`；单次取哈希 `O(1)`；空间 `O(n)`。调用者必须保证 `0<=l<=r<=n`。

4.2 子串相等、LCP 与字典序比较

同一原串内比较 `[l1,r1)` 与 `[l2,r2)`，先确认长度相等，再比较哈希。两条不同字符串若分别建立 `Hash`，因为共用 `BASE`，同样可直接比较等长子串的哈希。

```
bool same(const Hash& h, int l1, int r1, int l2, int r2) { // 同一原串的两段
    return r1 - l1 == r2 - l2 && h.get(l1, r1) == h.get(l2, r2);
}

int lcp(const Hash& a, int x, const Hash& b, int y, int lim) { // 最多比较 lim 个字符
    int l = 0, r = lim + 1;
    while (l + 1 < r) {
        int m = (l + r) >> 1;
        if (a.get(x, x + m) == b.get(y, y + m)) l = m;
        else r = m;
    }
    return l;
}
```

`lcp(a,x,b,y,lim)` 要求两串从 `x,y` 起都至少剩余 `lim` 个字符，返回 LCP 长度但不超过 `lim`，复杂度 `O(log lim)`。

比较两个子串字典序时，先二分 LCP；若其中一段耗尽，则短者小，否则比较下一个字符。大量后缀 LCP 查询用后缀数组会从 `O(log n)` 降到 `O(1)` 且无碰撞。

4.3 回文判断

给原串和反串分别建哈希。原串 `[l,r)` 在反串中对应 `[n-r,n-l)`。

```
struct PalHash {
    int n;
    string s, rs;
    Hash a, b;
    PalHash(const string& _s): n(_s.size()), s(_s), rs(_s.rbegin(), _s.rend()),
a(s), b(rs) {}
    bool ispal(int l, int r) const { // 判断 s[l,r) 是否回文
        return a.get(l, r) == b.get(n - r, n - l);
    }
};
```

预处理 `O(n)`，每次判回文期望 `O(1)`。若必须零碰撞，可使用 Manacher 预处理后 `O(1)` 判静态子串是否回文。

4.4 双模哈希（碰撞更低）

卡哈希风险较高或哈希值需要跨程序固定时，可使用两个质数模。它比 `ull` 慢，但结果可复现。

```
struct DHash {
    static const int M1 = 1000000007, M2 = 1000000009, B = 911382323;
    vector<int> a, b, p1, p2;
    DHash(const string& s = "") { init(s); }
    void init(const string& s) { // s: 原串
        int n = s.size();
        a.assign(n + 1, 0), b.assign(n + 1, 0);
        p1.assign(n + 1, 1), p2.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            int x = (unsigned char)s[i] + 1;
            a[i + 1] = ((ll)a[i] * B + x) % M1;
            b[i + 1] = ((ll)b[i] * B + x) % M2;
            p1[i + 1] = (ll)p1[i] * B % M1;
            p2[i + 1] = (ll)p2[i] * B % M2;
        }
    }
    pair<int,int> get(int l, int r) const { // 子串 [l,r)
        int x = (a[r] - (ll)a[l] * p1[r - l]) % M1;
        int y = (b[r] - (ll)b[l] * p2[r - l]) % M2;
        if (x < 0) x += M1;
        if (y < 0) y += M2;
        return {x, y};
    }
};
```

这里 `B<M1,M2` 且与模数互质。需要把哈希对作为 `unordered_map` 的键时，可打包成 `ull(x)<<32|y`。

4.5 哈希拼接与重复串

若 `A`、`B` 的哈希分别为 `ha`、`hb`，长度分别为 `la`、`lb`，则 `A+B` 的哈希为 `ha*BASE^lb+hb`。这能在不真正生成字符串时拼接区间：

```
ull hash_join(ull ha, ull hb, int lb, const vector<ull>& p) { // hash(A+B)
    return ha * p[lb] + hb;
}
```

比较不同来源的拼接串时，各段必须使用相同字符映射和 `BASE`。连续重复 `k` 次的哈希可以用二进制快速幂式拼接在 `O(log k)` 内求出；不能在模 `2^64` 下直接套等比数列除法，因为分母未必有逆元。

4.6 单点修改、区间哈希与动态判回文

线段树每个节点同时保存正向哈希、反向哈希和长度。支持单点改字符、查询任意 `[l,r)` 哈希、动态判回文，均为 `O(log n)`。这是概率算法；所有实例仍需共用前文的 `BASE`。

- `DynHash h(s)`：建立非空字符串。
- `set(i,c)`：把 `s[i]` 改为 `c`。
- `get(l,r)`：返回 `{正向哈希,反向哈希,长度}`。
- `ispal(l,r)`：判断 `[l,r)` 是否回文，空区间为真。

```
struct DynHash {
    struct H { ull f = 0, r = 0; int n = 0; };
    int n;
    vector<ull> p;
    vector<H> tr;

    DynHash(const string& s) { // s: 非空原串
        n = s.size();
        p.assign(n + 1, 1), tr.resize(4 * n);
        for (int i = 1; i <= n; i++) p[i] = p[i - 1] * BASE;
        build(1, 0, n, s);
    }
    H merge(const H& a, const H& b) const { // 表示串 a+b
        return {a.f * p[b.n] + b.f, b.r * p[a.n] + a.r, a.n + b.n};
    }
    void build(int u, int l, int r, const string& s) {
        if (r - l == 1) {
            ull x = (unsigned char)s[l] + 1;
            tr[u] = {x, x, 1};
            return;
        }
        int m = (l + r) >> 1;
        build(u << 1, l, m, s), build(u << 1 | 1, m, r, s);
        tr[u] = merge(tr[u << 1], tr[u << 1 | 1]);
    }
    void set(int u, int l, int r, int i, char c) {
        if (r - l == 1) {
            ull x = (unsigned char)c + 1;
            tr[u] = {x, x, 1};
            return;
        }
        int m = (l + r) >> 1;
        if (i < m) set(u << 1, l, m, i, c);
        else set(u << 1 | 1, m, r, i, c);
        tr[u] = merge(tr[u << 1], tr[u << 1 | 1]);
    }
    H get(int u, int l, int r, int ql, int qr) const {
        if (ql <= l && r <= qr) return tr[u];
        int m = (l + r) >> 1;
        if (qr <= m) return get(u << 1, l, m, ql, qr);
```

```
        if (ql >= m) return get(u << 1 | 1, m, r, ql, qr);
        return merge(get(u << 1, l, m, ql, qr),
                     get(u << 1 | 1, m, r, ql, qr));
    }
    void set(int i, char c) { set(1, 0, n, i, c); } // i: 0-based
    H get(int l, int r) const { // 区间 [l,r)，允许空串
        if (l == r) return {};
        return get(1, 0, n, l, r);
    }
    bool ispal(int l, int r) const {
        H x = get(l, r);
        return x.f == x.r;
    }
};
```

两个动态子串相等时，先比较长度，再比较各自 `get(l,r).f`。区间拼接顺序由 `merge` 保证，因此线段树查询不能改成无序聚合。

---

5. Manacher 回文算法

一次线性扫描求每个位置的奇回文和偶回文半径，不需要插入分隔符。

- `d1[i]`：以 `i` 为中心的奇回文半径，回文区间为 `[i-d1[i]+1, i+d1[i])`，长度 `2*d1[i]-1`。
- `d2[i]`：中心在 `i-1` 与 `i` 之间的偶回文半径，区间为 `[i-d2[i], i+d2[i])`，长度 `2*d2[i]`。
- 时间 `O(n)`，空间 `O(n)`。

```
struct Manacher {
    vector<int> d1, d2;
    Manacher(const string& s = "") { init(s); }
    void init(const string& s) { // s: 原串
        int n = s.size();
        d1.assign(n, 0), d2.assign(n, 0);
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
            while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
            d1[i] = k--;
            if (i + k > r) l = i - k, r = i + k;
        }
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
            while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
            d2[i] = k--;
            if (i + k > r) l = i - k - 1, r = i + k;
        }
    }
    bool ispal(int l, int r) const { // 判断 [l,r)，允许空串
        int len = r - l;
        if (!len) return true;
```

```
        if (len & 1) return d1[(l + r - 1) / 2] >= len / 2 + 1;
        return d2[(l + r) / 2] >= len / 2;
    }
};
```

`ispal(l,r)` 为零碰撞 `O(1)` 判回文，要求区间合法。空串会返回 `true`。

5.1 最长回文子串与回文总数

最长回文可扫描所有中心；所有回文子串个数（按位置区分）等于 `sum(d1)+sum(d2)`，可能达到 `n(n+1)/2`，必须使用 `ll`。

```
pair<int,int> longest_pal(const string& s) { // 返回最长回文的 [l,r)
    Manacher m(s);
    int L = 0, R = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        int l = i - m.d1[i] + 1, r = i + m.d1[i];
        if (r - l > R - L) L = l, R = r;
        l = i - m.d2[i], r = i + m.d2[i];
        if (r - l > R - L) L = l, R = r;
    }
    return {L, R};
}

ll count_pal(const string& s) { // 按位置区分的回文子串总数
    Manacher m(s);
    return accumulate(m.d1.begin(), m.d1.end(), ll(0))
         + accumulate(m.d2.begin(), m.d2.end(), ll(0));
}
```

本质不同回文串个数不能用上述求和，应使用回文自动机。

5.2 最长回文前后缀与最少补字符

Manacher 建好后可 `O(1)` 判区间，线性扫描即可求最长回文前缀/后缀。由此可求“只允许在一端添加字符，使整串成为回文串”的最短结果。

```
pair<int,int> pal_pre_suf(const string& s) { // 返回 {最长回文前缀长度,最长回文后缀长度}
    int n = s.size(), a = 0, b = 0;
    Manacher m(s);
    for (int r = n; r >= 0; r--) if (m.ispal(0, r)) { a = r; break; }
    for (int l = 0; l <= n; l++) if (m.ispal(l, n)) { b = n - l; break; }
    return {a, b};
}

string add_front_pal(const string& s) { // 只在开头补，返回最短回文串
    auto [p, q] = pal_pre_suf(s);
    string t = s.substr(p);
    reverse(t.begin(), t.end());
    return t + s;
```

```
}

string add_back_pal(const string& s) { // 只在末尾补，返回最短回文串
    auto [p, q] = pal_pre_suf(s);
    string t = s.substr(0, s.size() - q);
    reverse(t.begin(), t.end());
    return s + t;
}
```

总时间 `O(n)`，返回字符串本身需要 `O(n)` 空间。若只求最少添加字符数，答案分别是 `n-最长回文前缀长度` 和 `n-最长回文后缀长度`。

---

6. Trie 字典树

下面是小写字母 Trie，支持插入、精确计数、前缀计数和删除。节点池使用 `vector` 连续存储，缓存友好；总节点数不超过所有插入串长度之和加一。

- `insert(s)`：插入一次。
- `count(s)`：`s` 被完整插入的次数。
- `prefix_count(s)`：以 `s` 为前缀的已插入字符串数。
- `erase(s)`：删除一次；不存在则返回 `false`。为追求速度不回收节点。
- 每次操作时间 `O(|s|)`。

```
struct Trie {
    struct Node {
        int ch[26], pass, end;
        Node() { memset(ch, 0, sizeof ch); pass = end = 0; }
    };
    vector<Node> t;
    Trie() { t.emplace_back(); }
    void clear() { t.clear(); t.emplace_back(); }
    void reserve(int n) { t.reserve(n); } // n: 预计总节点数
    void insert(const string& s) { // s: 小写字母串
        int u = 0; t[u].pass++;
        for (char c : s) {
            int x = c - 'a';
            if (!t[u].ch[x]) t[u].ch[x] = t.size(), t.emplace_back();
            u = t[u].ch[x], t[u].pass++;
        }
        t[u].end++;
    }
    int count(const string& s) const { // 完整串出现次数
        int u = 0;
        for (char c : s) {
            u = t[u].ch[c - 'a'];
            if (!u) return 0;
        }
        return t[u].end;
```

```
    }
    int prefix_count(const string& s) const { // 以 s 为前缀的串数
        int u = 0;
        for (char c : s) {
            u = t[u].ch[c - 'a'];
            if (!u) return 0;
        }
        return t[u].pass;
    }
    bool erase(const string& s) { // 删除一次，不回收节点
        if (!count(s)) return false;
        int u = 0; t[u].pass--;
        for (char c : s) u = t[u].ch[c - 'a'], t[u].pass--;
        t[u].end--;
        return true;
    }
};
```

使用前若知道模式串总长度为 `L`，调用 `tr.reserve(L+1)` 可避免扩容。若 `pass/end` 可能超过 `int`，改为 `ll`。

字符集不是小写字母时，统一写一个映射函数，并把 `26` 改为字符集大小。例如大小写字母加数字为 `62`；完整 ASCII 可用 `128/256`，但节点数大时内存会显著增加。

6.1 字典序第 k 个字符串

已有 Trie 的 `pass/end` 支持按字典序取第 `k` 个已插入字符串，重复字符串按插入次数重复计算。字典序中一个串排在它的所有真扩展之前，因此到达节点后要先扣除 `end`。

```
string trie_kth(const Trie& a, int k) { // k: 1-based；越界返回空串
    if (k < 1 || k > a.t[0].pass) return "";
    string s;
    int u = 0;
    while (1) {
        if (k <= a.t[u].end) return s;
        k -= a.t[u].end;
        for (int c = 0; c < 26; c++) {
            int v = a.t[u].ch[c];
            if (!v) continue;
            if (k > a.t[v].pass) k -= a.t[v].pass;
            else { s += char('a' + c), u = v; break; }
        }
    }
}
```

时间 `O(答案长度*26)`；字符集较大时可维护有序孩子表。由于越界和“合法答案恰为空串”都会返回空串，若插入集合允许空串且需要区分，接口应改成 `optional<string>`。

---

7. AC 自动机

AC 自动机同时匹配多个模式串。本模板面向小写字母，支持重复模式串、统计每个模式串在一个或多个文本中的总出现次数，并用逆 BFS 沿 Fail 指针汇总，避免递归爆栈。

调用顺序必须是：全部 `insert` -> 一次 `build` -> 若干次 `query` -> 一次 `calc` -> `answer`。`build` 会补全 Trie 图转移，构建后不能再插入。

- `insert(s,id)`：插入非空模式串，`id` 可从 `0` 或 `1` 开始，但必须在 `pos` 范围内；重复串可用不同 `id`。
- `query(s)`：扫描一篇文本，允许重复调用以累计多篇文本。
- `calc()`：所有文本扫描完后调用一次；不可重复调用。
- `answer(id)`：模式串 `id` 的出现次数，允许重叠。
- 总复杂度：建机 `O(节点数*26)`，扫描 `O(文本总长)`，汇总 `O(节点数)`。

```
struct AC {
    struct Node {
        int ch[26], fail;
        ll cnt;
        Node() { memset(ch, 0, sizeof ch); fail = 0; cnt = 0; }
    };
    vector<Node> t;
    vector<int> pos, ord;
    AC(int m = 0) : t(1), pos(m) {} // m: 模式串编号空间
    void reserve(int n) { t.reserve(n); } // n: 预计总节点数
    void insert(const string& s, int id) { // s: 非空小写串; id: 模式编号
        int u = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!t[u].ch[x]) t[u].ch[x] = t.size(), t.emplace_back();
            u = t[u].ch[x];
        }
        pos[id] = u;
    }
    void build() { // 全部 insert 后调用一次
        queue<int> q;
        for (int c = 0; c < 26; c++)
            if (t[0].ch[c]) q.push(t[0].ch[c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            ord.push_back(u);
            for (int c = 0; c < 26; c++) {
                int v = t[u].ch[c];
                if (v) t[v].fail = t[t[u].fail].ch[c], q.push(v);
                else t[u].ch[c] = t[t[u].fail].ch[c];
            }
        }
    }
    void query(const string& s) { // 扫描文本，可调用多次
        int u = 0;
        for (char c : s) {
            if (c < 'a' || c > 'z') { u = 0; continue; }
```

```
            u = t[u].ch[c - 'a'];
            t[u].cnt++;
        }
    }
    void calc() { // 所有 query 后只调用一次
        for (int i = (int)ord.size() - 1; i >= 0; i--) {
            int u = ord[i];
            t[t[u].fail].cnt += t[u].cnt;
        }
    }
    ll answer(int id) const { return t[pos[id]].cnt; }
};
```

`cnt` 用 `ll`，因为“文本长度 × 文本篇数”可能超过 `int`。根的缺失转移保持为 `0`，非根缺失转移在 `build` 中被补成 Fail 状态的转移。

7.1 在线报告所有匹配位置

若必须输出每次匹配的位置，需要为每个终止节点保存模式编号，并建立 `out` 指针指向 Fail 链上最近的终止节点。下面模板直接返回三元组 `{id,l,r}`，其中匹配区间为 `[l,r)`。复杂度是 `O(|text|+输出总数)`；输出本身很多时不可能更快。

```
struct ACReport {
    struct Node {
        int ch[26]{}, fail = 0, out = 0;
        vector<int> ids;
    };
    vector<Node> t{1};
    vector<int> len;
    ACReport(int m): len(m) {} // m: 模式串编号空间
    void insert(const string& s, int id) { // s: 非空小写串
        int u = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!t[u].ch[x]) t[u].ch[x] = t.size(), t.emplace_back();
            u = t[u].ch[x];
        }
        t[u].ids.push_back(id), len[id] = s.size();
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < 26; c++) if (t[0].ch[c]) q.push(t[0].ch[c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int c = 0; c < 26; c++) {
                int v = t[u].ch[c];
                if (v) {
                    t[v].fail = t[t[u].fail].ch[c];
                    int f = t[v].fail;
                    t[v].out = !t[f].ids.empty() ? f : t[f].out;
                    q.push(v);
```

```
                } else t[u].ch[c] = t[t[u].fail].ch[c];
            }
        }
    }
    vector<array<int,3>> query(const string& s) const { // 返回 {id,l,r}
        vector<array<int,3>> ans;
        int u = 0;
        for (int i = 0; i < (int)s.size(); i++) {
            char c = s[i];
            if (c < 'a' || c > 'z') { u = 0; continue; }
            u = t[u].ch[c - 'a'];
            for (int v = u; v; v = t[v].out)
                for (int id : t[v].ids) ans.push_back({id, i - len[id] + 1, i + 1});
        }
        return ans;
    }
};
```

只统计各模式出现次数时应使用上一份逆 BFS 模板；逐个沿 `out` 链会为每个真实匹配付出时间，适合确实需要报告位置的题。

7.2 Fail 树与动态模式启用

把每个节点的 `fail[u]` 向 `u` 连边得到 Fail 树。模式串终止节点 `p` 能在当前自动机状态 `u` 处匹配，当且仅当 `p` 是 `u` 在 Fail 树上的祖先。这一结论可把许多 AC 问题转成树上问题：

- 扫描文本时在当前状态点加，模式出现次数等于其 Fail 子树点权和。
- 启用/停用一个模式节点 `p`，等价于给 Fail 子树 `[tin[p],tout[p])` 区间加 `+1/-1`；扫描到状态 `u` 时单点查询 `tin[u]`，得到此位置匹配的启用模式数。
- 在线加入文本字符并询问某模式累计出现次数：当前状态做 Euler 序单点加，询问模式节点的子树和。

构造 Euler 序可用迭代 DFS，避免 Fail 树退化时递归爆栈：

```
vector<pair<int,int>> fail_euler(const AC& a) { // 返回每个节点的 [tin,tout)
    int n = a.t.size(), tim = 0;
    vector<vector<int>> g(n);
    for (int u = 1; u < n; u++) g[a.t[u].fail].push_back(u);
    vector<pair<int,int>> d(n);
    vector<pair<int,int>> st{{0, 0}};
    d[0].first = tim++;
    while (!st.empty()) {
        int u = st.back().first, &i = st.back().second;
        if (i < (int)g[u].size()) {
            int v = g[u][i++];
            d[v].first = tim++;
            st.push_back({v, 0});
        } else {
            d[u].second = tim;
            st.pop_back();
```

```
        }
    }
    return d;
}
```

配合树状数组时，“子树区间加、单点查”可做差分：在 `tin` 加 `v`、在 `tout` 加 `-v`，查询前缀和；“单点加、子树和”则直接查询 `[tin,tout)` 区间和。重复模式串共用终止节点，若按编号启用，需要在同一节点累计启用数量。

---

8. 后缀数组 SA

后缀数组把所有后缀按字典序排列：`sa[k]` 是排名第 `k` 的后缀起点，`rk[i]` 是后缀 `s[i..)` 的排名。`ht[k]` 是 `sa[k]` 与 `sa[k-1]` 的 LCP，约定 `ht[0]=0`。

下面使用倍增加计数排序，构造时间 `O(n log n)`，空间 `O(n)`；常数较小且不依赖字符集。空串可正常构造。

```
struct SA {
    int n, K;
    vector<int> sa, rk, ht, lg;
    vector<vector<int>> st;

    SA(const string& s = "") { init(s); }

    void init(const string& s) { // s: 任意字节字符串
        n = s.size();
        sa.resize(n), rk.resize(n), ht.assign(n, 0);
        if (!n) { K = 0; return; }
        vector<int> x(n), y(n), c(max(256, n));
        for (int i = 0; i < n; i++) c[x[i] = (unsigned char)s[i]]++;
        for (int i = 1; i < 256; i++) c[i] += c[i - 1];
        for (int i = n - 1; i >= 0; i--) sa[--c[x[i]]] = i;
        for (int k = 1, m = 256, p = 0; k < n; k <<= 1, m = p) {
            p = 0;
            for (int i = n - k; i < n; i++) y[p++] = i;
            for (int i = 0; i < n; i++) if (sa[i] >= k) y[p++] = sa[i] - k;
            fill(c.begin(), c.begin() + m, 0);
            for (int i = 0; i < n; i++) c[x[y[i]]]++;
            for (int i = 1; i < m; i++) c[i] += c[i - 1];
            for (int i = n - 1; i >= 0; i--) sa[--c[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            for (int i = 1; i < n; i++) {
                int a = sa[i - 1], b = sa[i];
                bool eq = y[a] == y[b] && (a + k < n ? y[a + k] : -1) == (b + k < n
? y[b + k] : -1);
                x[b] = eq ? p - 1 : p++;
            }
            if (p == n) break;
        }
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
```

```
        for (int i = 0, k = 0; i < n; i++) {
            int r = rk[i];
            if (!r) continue;
            int j = sa[r - 1];
            while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
            ht[r] = k;
            if (k) k--;
        }
        build_rmq();
    }

    void build_rmq() { // 对 ht 建 ST 表
        if (!n) return;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i >> 1] + 1;
        K = lg[n] + 1;
        st.assign(K, vector<int>(n));
        st[0] = ht;
        for (int k = 1; k < K; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }

    int lcp(int i, int j) const { // 后缀 s[i..],s[j..] 的 LCP
        if (i == j) return n - i;
        int l = rk[i], r = rk[j];
        if (l > r) swap(l, r);
        l++;
        int k = lg[r - l + 1];
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

`lcp(i,j)` 要求 `0<=i,j<n`。ST 表额外占 `O(n log n)` 内存；若不需要任意后缀 LCP 查询，可删除 `lg/st/K`、`build_rmq()` 和 `lcp()`，仅保留 `sa/rk/ht`，空间即为 `O(n)`。内存紧且需要查询时可对 `ht` 建线段树，查询 `O(log n)`、空间 `O(n)`。

8.1 SA-IS 线性后缀数组

当 `n` 达到百万级、倍增的 `O(n log n)` 排序成为瓶颈时，可用 SA-IS 诱导排序。输入先映射为 `0..upper` 的整数；时间 `O(n+upper)`，空间 `O(n+upper)`。

`sais(s,upper)` 要求 `s[i]` 均在 `0..upper`。`sais_str(s)` 是任意字节字符串包装，返回值与前面的 `SA::sa` 相同。该模板只构造 `sa`；`rk/ht` 仍用前面 Kasai 的线性循环生成。

```
vector<int> sais(const vector<int>& s, int up) { // s[i] in [0,up]
    int n = s.size();
    if (!n) return {};
    if (n == 1) return {0};
    if (n == 2) return s[0] < s[1] ? vector<int>{0, 1} : vector<int>{1, 0};
```

```
    vector<int> sa(n), sl(up + 1), ss(up + 1);
    vector<bool> is(n);
    for (int i = n - 2; i >= 0; i--)
        is[i] = s[i] == s[i + 1] ? is[i + 1] : s[i] < s[i + 1];
    for (int i = 0; i < n; i++) {
        if (!is[i]) ss[s[i]]++;
        else sl[s[i] + 1]++;
    }
    for (int i = 0; i <= up; i++) {
        ss[i] += sl[i];
        if (i < up) sl[i + 1] += ss[i];
    }

    auto induce = [&](const vector<int>& lms) {
        fill(sa.begin(), sa.end(), -1);
        vector<int> b = ss;
        for (int x : lms) if (x < n) sa[b[s[x]]++] = x;
        b = sl;
        sa[b[s[n - 1]]++] = n - 1;
        for (int i = 0; i < n; i++) {
            int v = sa[i];
            if (v >= 1 && !is[v - 1]) sa[b[s[v - 1]]++] = v - 1;
        }
        b = sl;
        for (int i = n - 1; i >= 0; i--) {
            int v = sa[i];
            if (v >= 1 && is[v - 1]) sa[--b[s[v - 1] + 1]] = v - 1;
        }
    };

    vector<int> id(n + 1, -1), lms;
    int m = 0;
    for (int i = 1; i < n; i++)
        if (!is[i - 1] && is[i]) id[i] = m++;
    lms.reserve(m);
    for (int i = 1; i < n; i++) if (!is[i - 1] && is[i]) lms.push_back(i);
    induce(lms);

    if (m) {
        vector<int> a;
        a.reserve(m);
        for (int v : sa) if (id[v] != -1) a.push_back(v);
        vector<int> ns(m);
        int mx = 0;
        ns[id[a[0]]] = 0;
        for (int i = 1; i < m; i++) {
            int l = a[i - 1], r = a[i];
            int el = id[l] + 1 < m ? lms[id[l] + 1] : n;
            int er = id[r] + 1 < m ? lms[id[r] + 1] : n;
            bool eq = el - l == er - r;
            if (eq) {
                while (l < el && s[l] == s[r]) l++, r++;
```

```
                if (l == el && (l == n || s[l] == s[r])) {}
                else eq = false;
            }
            if (!eq) mx++;
            ns[id[a[i]]] = mx;
        }
        vector<int> nsa = sais(ns, mx), nlms(m);
        for (int i = 0; i < m; i++) nlms[i] = lms[nsa[i]];
        induce(nlms);
    }
    return sa;
}

vector<int> sais_str(const string& s) { // 任意字节串
    vector<int> a;
    a.reserve(s.size());
    for (unsigned char c : s) a.push_back(c);
    return sais(a, 255);
}
```

SA-IS 的代码明显长于倍增。通常 `n<=2e5` 时优先使用前面的倍增模板，调试风险更低；超长字符串或严格线性要求再使用本节。

8.2 常见结论

本质不同子串数量为所有子串数减去相邻后缀重复贡献：

```
ll distinct_substrings(const string& s) { // 本质不同非空子串数
    SA a(s);
    ll n = s.size();
    return n * (n + 1) / 2 - accumulate(a.ht.begin(), a.ht.end(), ll(0));
}
```

最长重复子串长度为 `max(ht)`。若要求至少出现 `k` 次的最长子串，在 `ht` 上取长度为 `k-1` 的滑动窗口最小值，再取最大值；`k=1` 时答案是 `n`。

```
int longest_repeat_k(const string& s, int k) { // 至少出现 k 次，允许重叠
    int n = s.size();
    if (k <= 1) return n;
    if (k > n) return 0;
    SA a(s);
    deque<int> q;
    int ans = 0;
    for (int i = 1; i < n; i++) {
        while (!q.empty() && a.ht[q.back()] >= a.ht[i]) q.pop_back();
        q.push_back(i);
        while (!q.empty() && q.front() <= i - k + 1) q.pop_front();
        if (i >= k - 1) ans = max(ans, a.ht[q.front()]);
    }
    return ans;
```

```
}
```

这里按起点不同计算出现，允许相互重叠。若题目要求出现位置互不重叠，仅看 LCP 窗口不够，还需在候选长度下检查后缀起点间距，常用二分答案加扫描。

两个子串 `[l1,r1)`、`[l2,r2)` 的 LCP 为 `min(sa.lcp(l1,l2), r1-l1, r2-l2)`。据此可零碰撞比较任意两段字典序。

8.3 两串最长公共子串

拼接 `a + 分隔符 + b` 建 SA。答案一定出现在来源不同的相邻后缀之间，取对应 `ht` 最大值即可。下面要求两串只含小写字母，以 `'{'` 作为唯一分隔符。

```
int lcs2_sa(const string& a, const string& b) { // a,b: 小写串；返回最长公共子串长度
    int n = a.size();
    string s = a + '{' + b;
    SA z(s);
    int ans = 0;
    auto side = [&](int x) { return x < n ? 0 : (x > n ? 1 : -1); };
    for (int i = 1; i < (int)s.size(); i++) {
        int x = side(z.sa[i - 1]), y = side(z.sa[i]);
        if (x >= 0 && y >= 0 && x != y) ans = max(ans, z.ht[i]);
    }
    return ans;
}
```

时间取决于所用 SA：倍增为 `O((n+m)log(n+m))`，SA-IS 为线性。任意字节串应转成整数数组并加入唯一分隔值，不能假设某个 `char` 一定未出现。多串版本需要在 SA 上维护覆盖所有来源的滑动窗口，并取窗口内 `ht` 的最小值。

8.4 查找模式串出现区间

所有以模式串 `p` 为前缀的后缀在 `sa` 中构成连续区间。可用两次二分找到区间，再把对应 `sa` 值作为出现位置。直接 `string::compare` 最坏会重复比较字符，复杂度约 `O(|p| log n + occ)`，一般足够；极限数据可结合 LCP 优化。

```
pair<int,int> sa_range(const string& s, const SA& a, const string& p) { // p: 非空模式；返回 SA 排名区间 [l,r)
    int n = s.size();
    auto cmp = [&](int pos) { return s.compare(pos, p.size(), p); };
    int l = 0, r = n;
    while (l < r) {
        int m = (l + r) >> 1;
        if (cmp(a.sa[m]) < 0) l = m + 1;
        else r = m;
    }
    int L = l; r = n;
    while (l < r) {
        int m = (l + r) >> 1;
        if (cmp(a.sa[m]) <= 0) l = m + 1;
```

```
        else r = m;
    }
    return {L, l};
}
```

若返回 `[L,R)`，则出现次数是 `R-L`，出现起点是 `sa[L..R)`。这些起点按后缀字典序而非数值顺序排列；需要位置递增时再排序。

---

9. 后缀自动机 SAM

SAM 的每个状态代表一组 `endpos` 相同的子串。对状态 `u`：

- `len[u]` 是该状态代表子串的最大长度。
- `link[u]` 指向其最长真后缀所属状态。
- 该状态新贡献 `len[u]-len[link[u]]` 个本质不同子串。
- 构造后按 `len` 降序把 `occ` 累加到 `link`，即可得到每个状态代表子串的出现次数。

下面固定小写字母，节点最多 `2n`，构造 `O(26n)` 的清零成本和 `O(n)` 的转移成本，空间 `O(26n)`。

```
struct SAM {
    struct Node {
        int ch[26], link, len, occ;
        Node() { memset(ch, 0, sizeof ch); link = -1; len = occ = 0; }
    };
    vector<Node> t;
    int last;
    SAM(int n = 0) { t.reserve(2 * n + 1); t.emplace_back(); last = 0; }
    int extend(char c) { // c: 新追加的小写字母
        int x = c - 'a', cur = t.size();
        t.emplace_back();
        t[cur].len = t[last].len + 1, t[cur].occ = 1;
        int p = last;
        while (p != -1 && !t[p].ch[x]) t[p].ch[x] = cur, p = t[p].link;
        if (p == -1) t[cur].link = 0;
        else {
            int q = t[p].ch[x];
            if (t[p].len + 1 == t[q].len) t[cur].link = q;
            else {
                int nq = t.size();
                t.push_back(t[q]);
                t[nq].len = t[p].len + 1, t[nq].occ = 0;
                while (p != -1 && t[p].ch[x] == q) t[p].ch[x] = nq, p = t[p].link;
                t[q].link = t[cur].link = nq;
            }
        }
        return last = cur;
    }
    void build(const string& s) { // 仅对新建 SAM 调用
        for (char c : s) extend(c);
```

```
    }
    vector<int> order() const { // 按 len 递增的状态编号
        int m = t.size(), mx = 0;
        for (auto &u : t) mx = max(mx, u.len);
        vector<int> c(mx + 1), o(m);
        for (auto &u : t) c[u.len]++;
        for (int i = 1; i <= mx; i++) c[i] += c[i - 1];
        for (int i = m - 1; i >= 0; i--) o[--c[t[i].len]] = i;
        return o;
    }
    void calc_occ() { // build 后调用一次
        auto o = order();
        for (int i = (int)o.size() - 1; i > 0; i--) {
            int u = o[i];
            t[t[u].link].occ += t[u].occ;
        }
    }
    int walk(const string& s) const { // 返回 s 所在状态，不是子串则 -1
        int u = 0;
        for (char c : s) {
            u = t[u].ch[c - 'a'];
            if (!u) return -1;
        }
        return u;
    }
    ll distinct() const { // 本质不同非空子串数
        ll ans = 0;
        for (int u = 1; u < (int)t.size(); u++) ans += t[u].len - t[t[u].link].len;
        return ans;
    }
};
```

典型用法：`SAM a(s.size()); a.build(s); a.calc_occ();`。随后模式 `p` 的出现次数是 `u=a.walk(p)` 后的 `a.t[u].occ`；若 `u==-1` 则为 `0`。`calc_occ()` 只能调用一次，否则会重复累加。

`walk(p)` 最终状态的 `occ` 就是 `p` 的出现次数，即使 `|p|<len[u]` 也成立，因为同一状态内所有长度大于 `len[link[u]]` 的后缀拥有相同 `endpos` 集合。

每个状态 `u` 恰好贡献长度区间 `[len[link[u]]+1,len[u]]` 中各一个本质不同子串。因此可用差分统计每种长度有多少个本质不同子串，也可用等差数列求所有不同子串长度之和：

```
vector<int> distinct_by_len(const SAM& a, int n) { // ans[len]: 不同子串个数
    vector<int> d(n + 2);
    for (int u = 1; u < (int)a.t.size(); u++) {
        int l = a.t[a.t[u].link].len + 1, r = a.t[u].len;
        d[l]++, d[r + 1]--;
    }
    for (int i = 1; i <= n; i++) d[i] += d[i - 1];
    d.resize(n + 1);
    return d;
}
```

```cpp
ll sum_distinct_len(const SAM& a) { // 所有本质不同非空子串的长度和
    ll ans = 0;
    for (int u = 1; u < (int)a.t.size(); u++) {
        ll l = a.t[a.t[u].link].len + 1, r = a.t[u].len;
        ans += (l + r) * (r - l + 1) / 2;
    }
    return ans;
}
```

所有不同子串的长度总和最坏为 `Theta(n^3)`，通常需 `ll`；若 `n` 极大还应判断是否可能超过 `ll`。

9.1 最长公共子串

先对 `a` 建 SAM，再扫描 `b`。返回两串最长公共**子串**长度，不是子序列。

```cpp
int lcs_substring(const SAM& a, const string& b) { // a: 已由第一串构建; b: 第二串
    int u = 0, l = 0, ans = 0;
    for (char c : b) {
        int x = c - 'a';
        while (u && !a.t[u].ch[x]) u = a.t[u].link, l = a.t[u].len;
        if (a.t[u].ch[x]) u = a.t[u].ch[x], l++;
        else u = 0, l = 0;
        ans = max(ans, l);
    }
    return ans;
}
```

复杂度 `O(|a|+|b|)`。若还需恢复子串，扫描时记录取得最大值时 `b` 的末位置。

9.2 第 k 小本质不同子串

令 `dp[u]` 为从状态 `u` 出发能形成的非空字符串数量，按字符转移即可做字典序第 `k` 小。每个本质不同子串只计一次。

```cpp
vector<ll> sam_dp(const SAM& a) { // dp[u]: 从 u 出发的非空路径数
    auto o = a.order();
    vector<ll> f(a.t.size());
    const ll LIM = (ll(1) << 62);
    for (int i = (int)o.size() - 1; i >= 0; i--) {
        int u = o[i];
        for (int c = 0; c < 26; c++) if (int v = a.t[u].ch[c])
            f[u] = min(LIM, f[u] + 1 + f[v]);
    }
    return f;
}

string kth_substring(const SAM& a, ll k) { // k: 1-based; 越界返回空串
    auto f = sam_dp(a);
    if (k < 1 || k > f[0]) return "";
    string s;
    int u = 0;
```

```
    while (k) {
        for (int c = 0; c < 26; c++) if (int v = a.t[u].ch[c]) {
            if (k > 1 + f[v]) k -= 1 + f[v];
            else {
                s += char('a' + c), u = v, k--;
                break;
            }
        }
    }
    return s;
}
```

`k` 从 `1` 开始；总数可能指数级，这里饱和到 `2^62`，足以支持 `ll` 范围内的询问。

9.3 出现至少 k 次的最长子串

执行 `calc_occ()` 后，状态 `u` 代表的最长串长度为 `len[u]`，出现次数为 `occ[u]`。因此出现至少 `k` 次的最长子串长度就是满足 `occ[u]>=k` 的最大 `len[u]`：

```
int sam_repeat_k(const SAM& a, int k) { // a: 已 calc_occ；至少出现 k 次
    int ans = 0;
    for (int u = 1; u < (int)a.t.size(); u++)
        if (a.t[u].occ >= k) ans = max(ans, a.t[u].len);
    return ans;
}
```

这里按不同起点计数并允许重叠。`k<=1` 时答案是原串长度。若要求不重叠出现，需要额外维护每个状态的结束位置集合或最早/最晚结束位置，不能只看 `occ`。

9.4 多串最长公共子串

先用第一串建立 SAM。对每个其他串扫描，`mx[u]` 记录本串能在状态 `u` 匹配的最大长度，再沿 Link 按长度降序传播；`mn[u]` 取所有串中的最小值。最终 `max(mn)` 是所有字符串的最长公共子串长度。

```
int lcs_many(const SAM& a, const vector<string>& vs) { // a: 第一串 SAM；vs: 其余小写串
    int m = a.t.size();
    vector<int> mn(m), mx(m);
    for (int u = 0; u < m; u++) mn[u] = a.t[u].len;
    auto o = a.order();
    for (const string& s : vs) {
        fill(mx.begin(), mx.end(), 0);
        int u = 0, l = 0;
        for (char c : s) {
            int x = c - 'a';
            while (u && !a.t[u].ch[x]) {
                u = a.t[u].link;
                l = min(l, a.t[u].len);
            }
            if (a.t[u].ch[x]) u = a.t[u].ch[x], l++;
            else u = 0, l = 0;
```

```
            mx[u] = max(mx[u], l);
        }
        for (int i = (int)o.size() - 1; i > 0; i--) {
            int v = o[i], p = a.t[v].link;
            mx[p] = max(mx[p], min(mx[v], a.t[p].len));
        }
        for (int u = 0; u < m; u++) mn[u] = min(mn[u], mx[u]);
    }
    return *max_element(mn.begin(), mn.end());
}
```

总时间 `O(第一串长度 + 其余串总长度 + 状态数*其余串数量)`。当字符串数量很多时，最后一项可能成为瓶颈，可只清空和更新本轮访问过的状态，或改用广义 SAM/后缀数组滑动窗口。

9.5 最短缺失子串

在 SAM 转移图上从根 BFS。到达状态 `u` 的路径都是原串子串；若某个字符转移不存在，则"该路径 + 字符"就是缺失子串。按字符从小到大扩展，可返回最短且字典序最小的缺失小写子串。

```
string shortest_absent(const SAM& a) { // 返回最短、字典序最小的缺失小写子串
    int n = a.t.size();
    vector<int> par(n, -1), pc(n, -1);
    queue<int> q;
    par[0] = -2, q.push(0);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int c = 0; c < 26; c++) {
            int v = a.t[u].ch[c];
            if (!v) {
                string s;
                for (int x = u; x; x = par[x]) s += char('a' + pc[x]);
                reverse(s.begin(), s.end());
                return s + char('a' + c);
            }
            if (par[v] == -1) par[v] = u, pc[v] = c, q.push(v);
        }
    }
    return "";
}
```

状态数最多 `2n`，复杂度 `O(26n)`。有限字符串必然存在缺失子串，因此正常情况下不会返回空串。

---

10. 回文自动机 PAM（回文树）

PAM 每个非根节点代表一个本质不同回文子串，因此建完后本质不同非空回文串数量是 `size-2`。两个根长度分别为 `-1` 和 `0`，这样奇偶回文可以统一转移。

下面模板支持：

- 在线向末尾追加小写字符。

- 得到每个前缀结尾处的最长回文后缀。
- 统计本质不同回文串数。
- `calc_occ()` 后得到每个回文串在整串中的出现次数。

构造时间 `O(n)` 均摊，节点最多 `n+2`，空间 `O(26n)`。

```
struct PAM {
    struct Node {
        int ch[26], link, len;
        ll occ;
        Node(int l = 0): link(0), len(l), occ(0) { memset(ch, 0, sizeof ch); }
    };
    vector<Node> t;
    string s;
    int last;
    PAM(int n = 0) {
        t.reserve(n + 2);
        t.emplace_back(-1), t.emplace_back(0);
        t[0].link = 0, t[1].link = 0, last = 1;
    }
    int get(int u, int i) const {
        while (i - 1 - t[u].len < 0 || s[i - 1 - t[u].len] != s[i]) u = t[u].link;
        return u;
    }
    int extend(char c) { // c: 新追加的小写字母
        s += c;
        int i = s.size() - 1, x = c - 'a', u = get(last, i);
        if (!t[u].ch[x]) {
            int v = t.size();
            t.emplace_back(t[u].len + 2);
            t[v].link = t[v].len == 1 ? 1 : t[get(t[u].link, i)].ch[x];
            t[u].ch[x] = v;
        }
        last = t[u].ch[x], t[last].occ++;
        return last;
    }
    void build(const string& a) { // 仅对新建 PAM 调用
        for (char c : a) extend(c);
    }
    void calc_occ() { // build 后调用一次
        int m = t.size(), n = s.size();
        vector<int> c(n + 1), o(m - 2);
        for (int u = 2; u < m; u++) c[t[u].len]++;
        for (int i = 1; i <= n; i++) c[i] += c[i - 1];
        for (int u = m - 1; u >= 2; u--) o[--c[t[u].len]] = u;
        for (int i = (int)o.size() - 1; i >= 0; i--) {
            int u = o[i];
            t[t[u].link].occ += t[u].occ;
        }
    }
    int distinct() const { return (int)t.size() - 2; }
```

```
};
```

`extend(c)` 返回新串的最长回文后缀所对应节点。节点 `u` 所代表回文串的长度是 `t[u].len`；若需要恢复内容，可在创建节点时额外记录其首次结束位置 `i`，对应区间是 `[i-len+1,i+1)`。

`calc_occ()` 前的 `occ[u]` 只表示构造过程中该节点作为最长回文后缀出现的次数；逆长度汇总后才是该回文串的总出现次数。该函数不可重复调用。

10.1 每个前缀的本质不同回文后缀个数

可给节点增加 `num`：`num[v]=num[link[v]]+1`。那么每次 `extend` 后 `num[last]` 就是当前前缀的本质不同回文后缀数量，累加所有前缀的该值等于按位置区分的回文子串总数。

在 `Node` 中加入 `int num=0;`，创建新节点后加入：

```
t[v].num = t[t[v].link].num + 1; // v: 新建回文节点
```

若只需要回文总数，Manacher 更短、常数更小；需要回文种类、出现次数或在线追加时用 PAM。

建树后的常用结论：

- 最长本质不同回文串长度为所有非根节点 `len` 的最大值。
- `calc_occ()` 后，出现次数最多的回文串取 `occ` 最大值；题目常问的“长度乘出现次数最大值”为 `max(len[u]*occ[u])`，乘法用 `ll`。
- Fail 链依次给出一个回文串的所有回文后缀。
- 节点创建时记录首次结束位置，即可恢复该节点代表的具体回文串。

---

11. Lyndon 分解与 Duval 算法

Lyndon 串是严格小于其所有非平凡后缀的字符串。Duval 算法把字符串唯一分解为字典序非增的 Lyndon 串序列，常用于字符串字典序结构、最小表示法和 Lyndon 分解题。

`duval(s)` 返回每一段的左闭右开区间 `[l,r)`，这些段依次拼接回原串。时间 `O(n)`，除返回值外空间 `O(1)`。

```
vector<pair<int,int>> duval(const string& s) { // 返回 Lyndon 分解的 [l,r)
    int n = s.size();
    vector<pair<int,int>> a;
    for (int i = 0; i < n; ) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) {
            a.push_back({i, i + j - k});
            i += j - k;
        }
```

```
    }
    return a;
}
```

例如需要输出各因子，可遍历返回区间并取 `s.substr(l,r-l)`。若仅求最小循环表示，前面的 Booth 模板接口更直接。

---

12. 子序列自动机

对固定主串预处理 `nxt[i][c]`：从位置 `i` 开始，第一个字符 `c` 出现在哪里。之后每个询问串可在线性时间判断是否为主串的子序列，并可恢复最早匹配位置。

下面针对小写字母，预处理时间和空间 `O(26n)`，单次询问 `O(m)`。

```
struct Subseq {
    int n;
    vector<array<int,26>> nx;
    Subseq(const string& s = "") { init(s); }
    void init(const string& s) { // s: 小写主串
        n = s.size();
        nx.resize(n + 1);
        nx[n].fill(n);
        for (int i = n - 1; i >= 0; i--) {
            nx[i] = nx[i + 1];
            nx[i][s[i] - 'a'] = i;
        }
    }
    vector<int> match(const string& t) const { // t: 询问串; 返回最早匹配位置
        vector<int> a;
        int p = 0;
        for (char c : t) {
            if (p > n || nx[p][c - 'a'] == n) return {};
            p = nx[p][c - 'a'];
            a.push_back(p++);
        }
        return a;
    }
    bool contains(const string& t) const { // 空串返回 true
        if (t.empty()) return true;
        return !match(t).empty();
    }
};
```

`match(t)` 成功时返回 `t` 每个字符在主串中的最早可行位置；失败返回空数组。因为空询问成功也返回空数组，所以 `contains` 对空串单独处理。

字符集很大或内存受限时，不开 `nxt` 表，而是为每个字符保存有序位置数组，每个字符用 `upper_bound` 找下一个位置。预处理 `O(n)`，询问 `O(m log n)`，空间 `O(n)`：

```
struct SubseqPos {
```

```
    array<vector<int>,256> p;
    SubseqPos(const string& s = "") {
        for (int i = 0; i < (int)s.size(); i++) p[(unsigned char)s[i]].push_back(i);
    }
    bool contains(const string& t) const { // t: 任意字节询问串
        int at = -1;
        for (unsigned char c : t) {
            auto it = upper_bound(p[c].begin(), p[c].end(), at);
            if (it == p[c].end()) return false;
            at = *it;
        }
        return true;
    }
};
```

---

13. LCS 最长公共子序列

注意 LCS 是子序列，可以不连续；最长公共子串必须连续，宜用 SAM、SA 或哈希二分。

13.1 滚动数组 LCS

适合 `nm` 可承受且只求长度。为了节省内存，令第二个串为较短串。

```
int lcs(const string& a, const string& b) { // 返回 LCS 长度
    const string *x = &a, *y = &b;
    if (x->size() < y->size()) swap(x, y);
    int m = y->size();
    vector<int> f(m + 1), g(m + 1);
    for (char c : *x) {
        for (int j = 1; j <= m; j++)
            g[j] = c == (*y)[j - 1] ? f[j - 1] + 1 : max(f[j], g[j - 1]);
        swap(f, g);
    }
    return f[m];
}
```

时间 `O(nm)`，空间 `O(min(n,m))`。代码中的 `g[0]` 始终为 `0`；每行其他位置都会被覆盖。

若要恢复一个具体 LCS，需保留完整 DP 表后从 `(n,m)` 倒推，空间 `O(nm)`；或使用 Hirschberg 算法在 `O(nm)` 时间、`O(min(n,m))` 空间恢复。

13.2 bitset LCS：`O(nm/64)`

当字符集较小、只求长度且 `m` 很大时，用位并行显著提速。下面是动态 `ull` 实现，不受 `std::bitset` 编译期长度限制，支持任意字节字符串。

- `a,b`：两个字符串，内部自动让 `b` 较短以节省内存。
- 返回：LCS 长度。
- 令 `b` 为较短串，时间 `O(256|b|/64+|a||b|/64)`，空间约 `O(256|b|/64)`。

```cpp
int lcs_bit(string a, string b) { // 返回 LCS 长度，任意字节串
    if (a.size() < b.size()) swap(a, b);
    int m = b.size(), W = (m + 63) >> 6;
    vector<vector<ull>> p(256, vector<ull>(W));
    for (int i = 0; i < m; i++) p[(unsigned char)b[i]][i >> 6] |= 1ULL << (i & 63);
    vector<ull> s(W), x(W), y(W), z(W);
    for (unsigned char c : a) {
        for (int k = 0; k < W; k++) x[k] = s[k] | p[c][k];
        ull carry = 1;
        for (int k = 0; k < W; k++) {
            ull nc = s[k] >> 63;
            y[k] = (s[k] << 1) | carry;
            carry = nc;
        }
        ull borrow = 0;
        for (int k = 0; k < W; k++) {
            ull q = y[k] + borrow;
            ull b1 = q < y[k];
            z[k] = x[k] - q;
            ull b2 = x[k] < q;
            borrow = b1 | b2;
        }
        for (int k = 0; k < W; k++) s[k] = x[k] & ~z[k];
        if (m & 63) s.back() &= (1ULL << (m & 63)) - 1;
    }
    int ans = 0;
    for (ull v : s) ans += __builtin_popcountll(v);
    return ans;
}
```

核心状态是位集形式的 DP 差分，更新公式为 `S = X & ~(X-(S<<1|1))`。手写多字长减法时必须正确传播借位；上面已处理 `y+borrow` 自身溢出的情况。空串时 `W=0`，循环不会访问 `s.back()`，返回 `0`。

13.3 排列 LCS 转 LIS

若两序列元素各不相同，且两序列包含同一集合，把第一序列元素映射为位置，再将第二序列替换成位置序列，LCS 就是该序列的 LIS。复杂度从 `O(n^2)` 降至 `O(n log n)`。

```cpp
template<class T>
int lcs_perm(const vector<T>& a, const vector<T>& b) { // a,b: 无重复且元素集合相同
    map<T,int> p;
    for (int i = 0; i < (int)a.size(); i++) p[a[i]] = i;
    vector<int> d;
    for (auto &x : b) {
        int v = p[x];
        auto it = lower_bound(d.begin(), d.end(), v);
        if (it == d.end()) d.push_back(v);
        else *it = v;
    }
    return d.size();
}
```

若值是 `1..n` 的排列，用 `vector<int> p(n+1)` 替代 `map`，总复杂度为 `O(n log n)` 且常数更小。存在重复元素时不能直接这样映射，可用 Hunt-Szymanski：按逆序枚举匹配位置，再做 LIS。

13.4 Hunt-Szymanski 稀疏匹配 LCS

令 `r` 为两串相等字符位置对数量。当字符集大、相同字符对很少时，枚举每个字符在第二串中的位置，并按逆序加入 LIS，复杂度 `O((n+r)log m)`，常比 `O(nm)` DP 快；最坏 `r=nm` 时反而不适用。

```cpp
int lcs_sparse(const string& a, const string& b) { // 任意字节串；只返回长度
    array<vector<int>,256> p;
    for (int i = 0; i < (int)b.size(); i++) p[(unsigned char)b[i]].push_back(i);
    vector<int> d;
    for (unsigned char c : a) {
        for (int k = (int)p[c].size() - 1; k >= 0; k--) {
            int x = p[c][k];
            auto it = lower_bound(d.begin(), d.end(), x);
            if (it == d.end()) d.push_back(x);
            else *it = x;
        }
    }
    return d.size();
}
```

对每个 `a[i]` 必须逆序枚举 `b` 中的匹配位置，否则同一个 `a[i]` 可能在 LIS 中被使用多次。若输入是整数序列，把 `array<...,256>` 换成离散化后的 `vector<vector<int>>`。

13.5 恢复一个 LCS

数据规模允许 `O(nm)` 内存时，保存完整 DP 并倒推最稳妥。相等时选该字符；不等时走值较大的相邻状态。多个最优解存在时，本模板返回其中任意一个，不保证字典序最小。

```cpp
string lcs_restore(const string& a, const string& b) { // 返回一个 LCS
    int n = a.size(), m = b.size();
    vector<vector<int>> f(n + 1, vector<int>(m + 1));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
```

```
            f[i][j] = a[i - 1] == b[j - 1] ? f[i - 1][j - 1] + 1
                                           : max(f[i - 1][j], f[i][j - 1]);
    string s;
    for (int i = n, j = m; i && j; ) {
        if (a[i - 1] == b[j - 1]) s += a[i - 1], i--, j--;
        else if (f[i - 1][j] >= f[i][j - 1]) i--;
        else j--;
    }
    reverse(s.begin(), s.end());
    return s;
}
```

时间和空间均为 `O(nm)`。内存不足但必须恢复时使用 Hirschberg 分治：正反各做一次滚动 DP 决定切分点，保持 `O(nm)` 时间，把额外空间降为 `O(min(n,m))`。

---

14. 编辑距离

Levenshtein 距离允许插入、删除、替换一个字符，求把 `a` 变成 `b` 的最少操作数。下面只求距离，自动让第二维较短。

```
int edit_dist(const string& a, const string& b) { // 插入/删除/替换代价均为 1
    const string *x = &a, *y = &b;
    if (x->size() < y->size()) swap(x, y);
    int m = y->size();
    vector<int> f(m + 1), g(m + 1);
    iota(f.begin(), f.end(), 0);
    for (int i = 1; i <= (int)x->size(); i++) {
        g[0] = i;
        for (int j = 1; j <= m; j++)
            g[j] = min({f[j] + 1, g[j - 1] + 1,
                        f[j - 1] + ((*x)[i - 1] != (*y)[j - 1])});
        swap(f, g);
    }
    return f[m];
}
```

时间 `O(nm)`，空间 `O(min(n,m))`。若操作代价不同，把三项分别改成删除、插入、替换代价；如果要恢复方案，需保留完整 DP 或使用分治恢复。

当只关心距离是否不超过较小阈值 `k` 时，只计算主对角线附近宽度 `2k+1` 的带状区域，可降到 `O(k(n+m))`；若某一行最小值已大于 `k` 可提前退出。

14.1 Myers 位并行编辑距离（短串不超过 64）

把较短串的一整列 DP 压入一个 `ull`，每扫描长串一个字符做常数次位运算。适合大量短模式与长文本比较。

- 自动把 `a` 调整为较短串，要求 `min(|a|,|b|)<=64`。
- 返回完整 Levenshtein 距离，不是近似值。

- 时间 `O(max(n,m))`，空间 `O(256)`。

```
int edit64(string a, string b) { // 要求 min(|a|,|b|)<=64
    if (a.size() > b.size()) swap(a, b);
    int n = a.size();
    if (!n) return b.size();
    assert(n <= 64);
    array<ull,256> eq{};
    for (int i = 0; i < n; i++) eq[(unsigned char)a[i]] |= 1ULL << i;
    ull pv = ~0ULL, mv = 0, top = 1ULL << (n - 1);
    int ans = n;
    for (unsigned char c : b) {
        ull e = eq[c], xv = e | mv;
        ull xh = (((e & pv) + pv) ^ pv) | e;
        ull ph = mv | ~(xh | pv), mh = pv & xh;
        if (ph & top) ans++;
        else if (mh & top) ans--;
        ph = (ph << 1) | 1;
        mh <<= 1;
        pv = mh | ~(xv | ph);
        mv = ph & xv;
    }
    return ans;
}
```

长度恰为 `64` 时移位仍合法，因为最高位使用 `1ULL<<63`。更长模式可扩展为多块 Myers，复杂度 `O(max(n,m)*ceil(min(n,m)/64))`，但进位和分数维护较易写错；只做一次时前面的普通 DP 更稳。

---

15. 字符串拼接与常用 DP

15.1 最大后缀—前缀重叠

求最大的 `k`，使 `a` 的长度 `k` 后缀等于 `b` 的长度 `k` 前缀。它用于按给定顺序压缩字符串、构造超串和处理首尾衔接。下面把字符转成整数并加入 `-1` 分隔符，支持任意字节字符串。

```
int overlap(const string& a, const string& b) { // max k: suffix(a,k)==prefix(b,k)
    int k = min(a.size(), b.size());
    vector<int> s;
    s.reserve(b.size() + k + 1);
    for (unsigned char c : b) s.push_back(c);
    s.push_back(-1);
    for (int i = (int)a.size() - k; i < (int)a.size(); i++)
        s.push_back((unsigned char)a[i]);
    vector<int> p(s.size());
    for (int i = 1; i < (int)s.size(); i++) {
        int j = p[i - 1];
        while (j && s[i] != s[j]) j = p[j - 1];
        if (s[i] == s[j]) j++;
        p[i] = j;
```

```
    }
    return p.empty() ? 0 : p.back();
}

string merge_overlap(const string& a, const string& b) { // 按 a 在前、b 在后合并
    return a + b.substr(overlap(a, b));
}
```

时间 `O(|a|+|b|)`，返回合并串还需与答案等长的空间。依次合并一列字符串只保证该给定顺序下最短；顺序可调整时，贪心选择最大重叠不保证全局最优。

15.2 最短公共超串（小规模）

给定少量字符串，求包含每个输入串作为子串的最短字符串。先删除被其他串包含的字符串，再用 `dp[mask][i]` 表示已经放入 `mask` 且最后一个串是 `i` 时的最短长度。

- 适合去重后 `n<=18~20`。
- 返回一个最短超串；存在多个答案时不保证字典序最小。
- 时间 `O(n^2*2^n + n^2L)`，空间 `O(n*2^n)`。

```
string shortest_superstring(vector<string> a) { // a: 少量任意字节串
    int z = a.size();
    vector<char> del(z);
    for (int i = 0; i < z; i++)
        for (int j = 0; j < z; j++) if (i != j && a[j].find(a[i]) != string::npos)
            if (a[j].size() > a[i].size() || (a[j] == a[i] && j < i)) del[i] = 1;
    vector<string> b;
    for (int i = 0; i < z; i++) if (!del[i]) b.push_back(a[i]);
    int n = b.size();
    if (!n) return "";
    vector<vector<int>> ov(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) if (i != j) ov[i][j] = overlap(b[i], b[j]);

    int S = 1 << n, INF = 1e9;
    vector<vector<int>> f(S, vector<int>(n, INF));
    vector<vector<signed char>> pre(S, vector<signed char>(n, -1));
    for (int i = 0; i < n; i++) f[1 << i][i] = b[i].size();
    for (int s = 1; s < S; s++) for (int i = 0; i < n; i++) if (s >> i & 1) {
        if (f[s][i] == INF) continue;
        for (int j = 0; j < n; j++) if (!(s >> j & 1)) {
            int ns = s | 1 << j;
            int v = f[s][i] + (int)b[j].size() - ov[i][j];
            if (v < f[ns][j]) f[ns][j] = v, pre[ns][j] = i;
        }
    }
    int s = S - 1, u = min_element(f[s].begin(), f[s].end()) - f[s].begin();
    vector<int> o;
    while (u != -1) {
        o.push_back(u);
```

```
        int v = pre[s][u];
        s ^= 1 << u, u = v;
    }
    reverse(o.begin(), o.end());
    string ans = b[o[0]];
    for (int i = 1; i < n; i++) ans += b[o[i]].substr(ov[o[i - 1]][o[i]]);
    return ans;
}
```

`signed char` 存前驱只适用于 `n<=127`，而子集 DP 实际远达不到该上限。若要求最短超串中字典序最小，需要在长度相同的转移间比较完整候选顺序，不能只靠较小前驱编号。

15.3 最少回文划分

把整串切成最少个回文子串。Manacher 预处理后枚举所有 `[l,r)`，判回文为 `O(1)`。

```
int min_pal_parts(const string& s) { // 返回最少回文段数；空串返回 0
    int n = s.size();
    Manacher m(s);
    vector<int> f(n + 1, n + 1);
    f[0] = 0;
    for (int r = 1; r <= n; r++)
        for (int l = 0; l < r; l++)
            if (m.ispal(l, r)) f[r] = min(f[r], f[l] + 1);
    return f[n];
}
```

时间 `O(n^2)`，空间 `O(n)`。若题目问最少切割次数，非空串答案为 `min_pal_parts(s)-1`，空串为 `0`。`n` 达到 `1e5` 时需使用带 `diff/slink` 的回文自动机优化，普通二次 DP 不再适用。

15.4 不同子序列数量

`f[i]` 表示前 `i` 个字符能形成的本质不同子序列数，包含空串。追加字符时原有子序列可选或不选，先乘二；若该字符上次在位置 `p` 出现，则减去 `f[p-1]` 消除重复。

```
int distinct_subseq(const string& s, int mod) { // 含空串；mod: 正整数
    int n = s.size();
    vector<int> f(n + 1);
    array<int,256> last{}; // 上次位置，1-based；0 表示未出现
    f[0] = 1 % mod;
    for (int i = 1; i <= n; i++) {
        int c = (unsigned char)s[i - 1];
        f[i] = (ll)f[i - 1] * 2 % mod;
        if (last[c]) {
            f[i] -= f[last[c] - 1];
            if (f[i] < 0) f[i] += mod;
        }
        last[c] = i;
    }
    return f[n];
```

```
}
```

时间 `O(n)`，空间 `O(n+256)`；只保留答案但仍要扣除历史 `f[p-1]`，所以需要保存 `f`。非空不同子序列数为 `(ans-1+mod)%mod`。不取模时数量可达 `2^n`，很快超过 `ll`。

16. BWT（Burrows-Wheeler 变换）

BWT 把字符串重排为更利于压缩和 FM-index 的形式，本身可逆。给原串追加唯一哨兵 `$`，建立后缀数组；每个后缀在循环移位中的前一个字符组成 BWT。

下面要求原串不含 `$`。`bwt(s)` 返回包含一个 `$` 的变换串；`ibwt(t)` 恢复不含哨兵的原串。任意二进制数据应改用整数数组和独立哨兵。

```
string bwt(const string& s) { // s 中不得含 '$'
    string t = s + '$', z;
    SA a(t);
    z.reserve(t.size());
    for (int p : a.sa) z += p ? t[p - 1] : t.back();
    return z;
}

string ibwt(const string& s) { // s: 合法 BWT，恰含一个 '$'
    int n = s.size();
    array<int,256> cnt{}, st{};
    vector<int> rk(n), lf(n);
    for (int i = 0; i < n; i++) rk[i] = cnt[(unsigned char)s[i]]++;
    for (int c = 1; c < 256; c++) st[c] = st[c - 1] + cnt[c - 1];
    int row = -1;
    for (int i = 0; i < n; i++) {
        lf[i] = st[(unsigned char)s[i]] + rk[i];
        if (s[i] == '$') row = i;
    }
    string t;
    for (int i = 1; i < n; i++) row = lf[row], t += s[row];
    reverse(t.begin(), t.end());
    return t;
}
```

构造时间取决于 SA；逆变换时间和空间均为 `O(n+256)`。BWT 不是哈希，也不会丢失信息；真正用于快速子串计数时，还需在其上建立 `occ`/小波矩阵形成 FM-index。

17. 常用字符串技巧

17.1 字符串去重与计数

需要对大量完整字符串去重时，优先直接排序，确定性强且通常比哈希表稳定：

```
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end()); // a: vector<string>
```

总复杂度取决于字符串比较字符数。若需要在线计数，用 `unordered_map<string,int>`；数据可能卡哈希时使用 `map` 或自定义随机哈希。

17.2 多串安全拼接

对多个字符串使用 SA/Z/Manacher 等算法时，分隔符必须互不相同且不出现在原字符集中，否则匹配可能跨串。字节字符不够用时转为 `vector<int>`，先把原字符离散到正整数，再使用 `0,-1,-2,...` 作为不同分隔符。

17.3 子串数量上界

长度为 `n` 的串共有 `n(n+1)/2` 个非空子串，可能超过 `int`；计数必须用 `ll`。本质不同子串最多也是该数量，SAM/SA 的答案同样使用 `ll`。

17.4 输入含空格

`cin >> s` 只读到空白处。整行字符串使用：

```
string s;
getline(cin, s); // 前面若刚用过 cin >> x，先 getline 吃掉行末或用 cin.ignore()
```

17.5 算法辨析

- 子串必须连续，子序列可以不连续。
- 最长回文子串用 Manacher；本质不同回文串用 PAM。
- 单模式匹配用 KMP/Z；多模式匹配用 AC。
- 静态任意后缀 LCP 且要求零碰撞用 SA；动态向末尾加字符、统计子串用 SAM。
- 哈希接口最灵活，但答案正确性是概率性的。
- `n` 很大时先估内存：`int ch[26]` 每个节点约 `104` 字节，`2n` 个 SAM 状态会很快超过内存限制。字符集稀疏时改用稀疏转移。

17.6 更高级模块的定位

以下算法没有再放完整长模板，但应知道何时使用：

- **广义 SAM**：大量字符串共同建自动机、统计跨串子串信息；只求多串最长公共子串时，本板 `lcs_many` 或 SA 滑窗更短。
- **后缀树**：可在线性空间表达所有后缀和 LCP 区间；多数竞赛功能可由 SA+LCP 或 SAM 更稳地完成。
- **Runs / Main-Lorentz**：在线性或 `O(n log n)` 时间枚举所有极大周期段；题目明确要求所有重复区间、串联重复时才值得使用。
- **PAM series link**：把最少回文划分等回文 DP 优化到接近线性；`n` 很大且二次 DP 超时时使用。
- **FM-index**：BWT 上叠加 `rank/Occ`（常用小波矩阵），支持压缩空间中的模式计数；普通内存限制下 SA 区间二分更简单。

- **通配符匹配卷积**：含 `?` 的模式匹配可用 NTT/FFT 构造失配卷积，相关多项式代码复用 `NTT FFT板子.md`。
- **多模式禁串计数**：AC 自动机状态作为 DP 状态；单模式禁串用 KMP 自动机更轻。

---

18. 赛前自检

1. 确认题目位置是 `0-based` 还是 `1-based`，输出时统一转换。
2. 确认区间是 `[l,r)` 还是 `[l,r]`；本板全部使用 `[l,r)`。
3. KMP 模式串、AC/PAM/SAM 的字符集是否满足模板假设。
4. AC 必须先全部插入再 `build`；`calc` 只能执行一次。
5. SAM/PAM 的出现次数汇总也只能执行一次。
6. 哈希比较的字符串必须共用基数；回文反串区间是 `[n-r,n-l)`。
7. SA 的 `lcp(i,j)` 不接受 `i==n`；空后缀若需要参与排序要显式加入哨兵。
8. 所有子串数、出现次数总和、答案乘积使用 `ll`。
9. 输出所有匹配时，复杂度必须加上输出总数。
10. 节点池按“所有插入字符总数 + 根”估算，SAM 最坏按 `2n` 估算。